

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللہی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 5/30

Computation, Initialization, Assignment, and Statements (Part 1)

- Initialization vs. Assignment
- Initialization and C++11 uniform initialization
- Writing simple arithmetic programs
- Expressions and statements
- Autos: C++11 automatic type deduction
- The elements of structured programming
- References and further readings
- Q&A

120 min (incl. Q & A)



I nitialization vs. assignment

I nitialization vs. assignment

- Initialization differs from assignment.

Initialization vs. assignment

- Initialization differs from assignment.

WS.

Initialization vs. assignment

- Initialization differs from assignment.

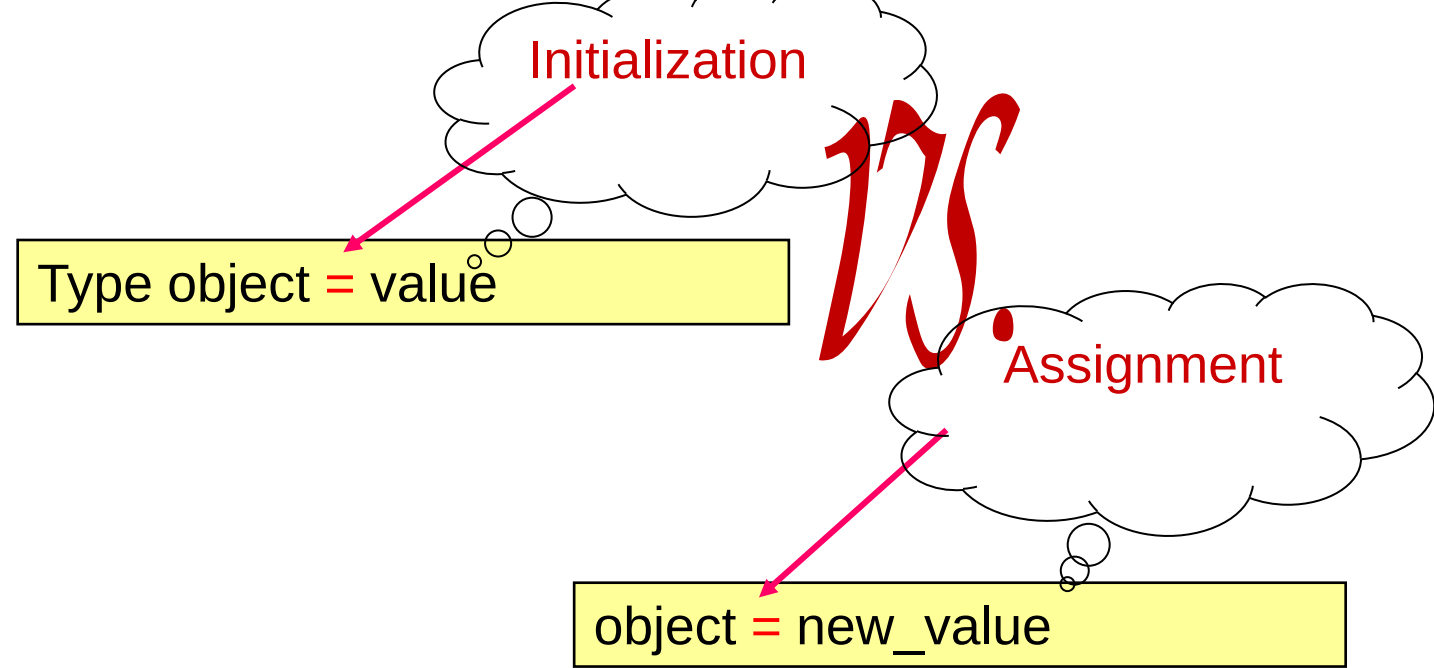
Type object = value

vs.

object = new_value

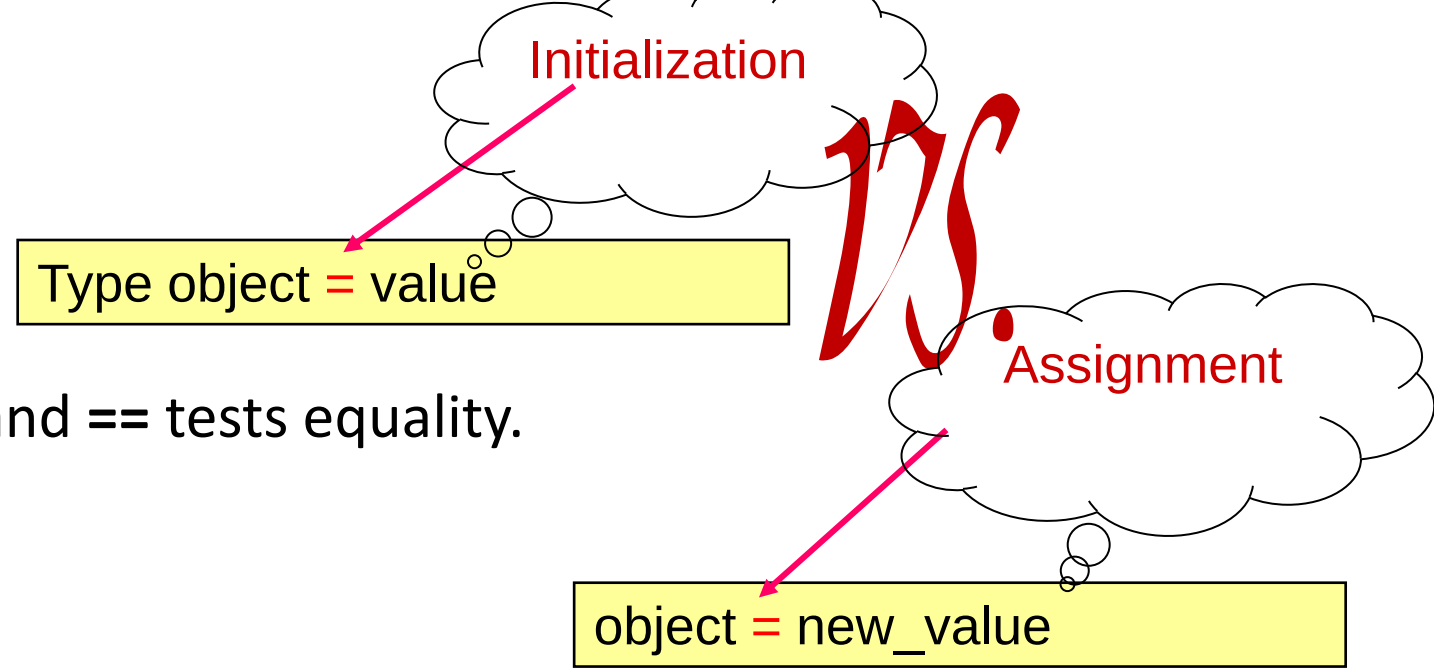
Initialization vs. assignment

- Initialization differs from assignment.



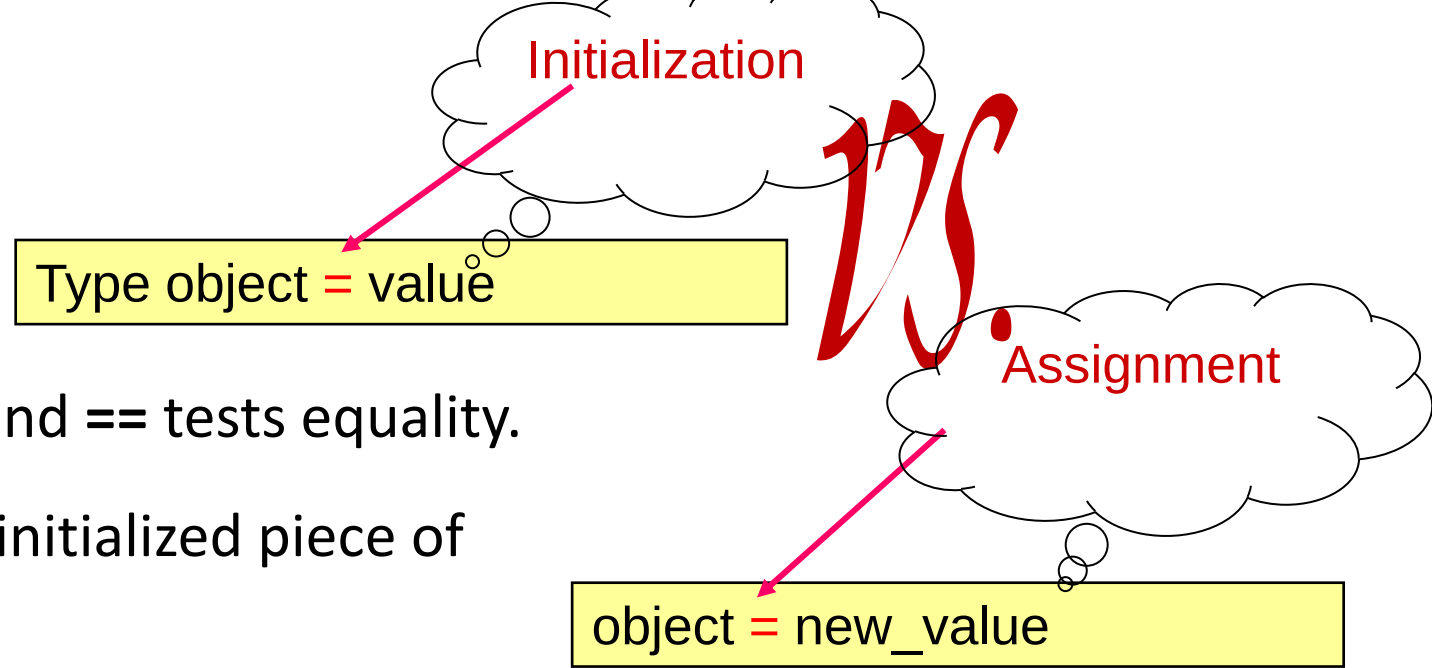
Initialization vs. assignment

- Initialization differs from assignment.
- Note that `=` is the assignment operator and `==` tests equality.



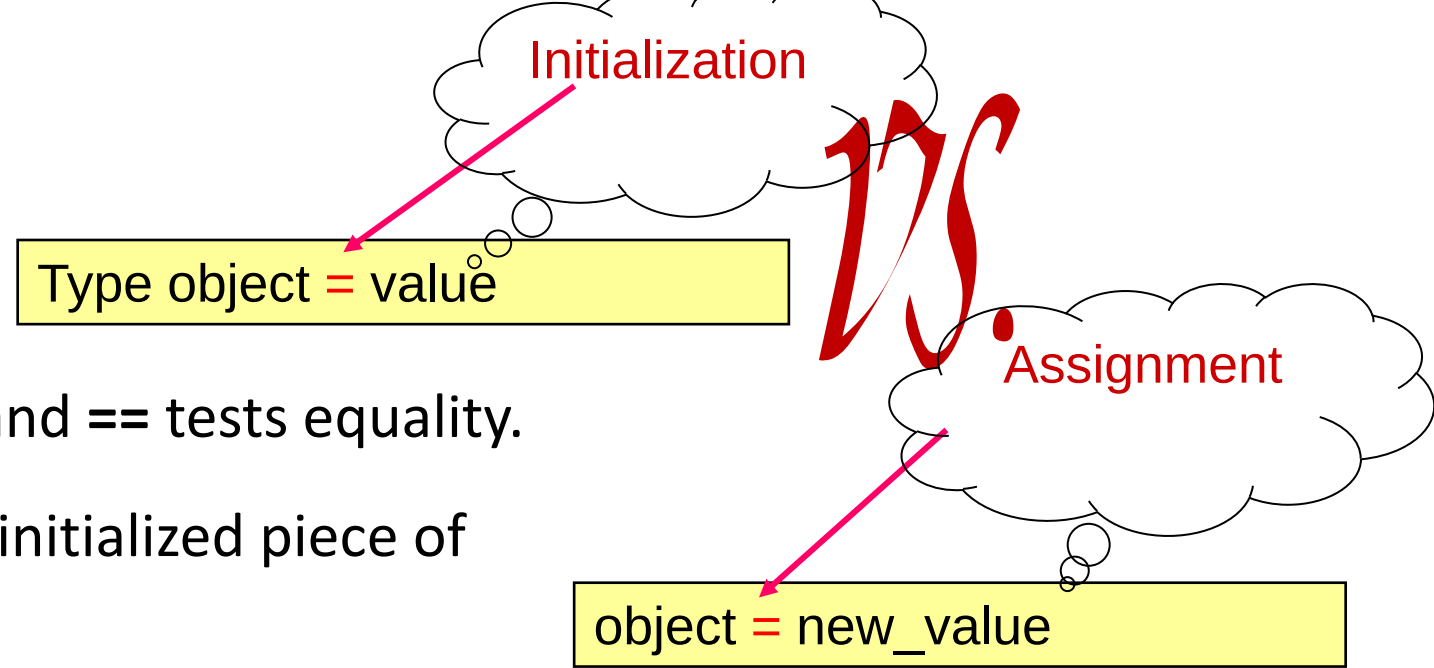
Initialization vs. assignment

- Initialization differs from assignment.
- Note that `=` is the assignment operator and `==` tests equality.
- the task of initialization is to make an uninitialized piece of memory into a valid object.



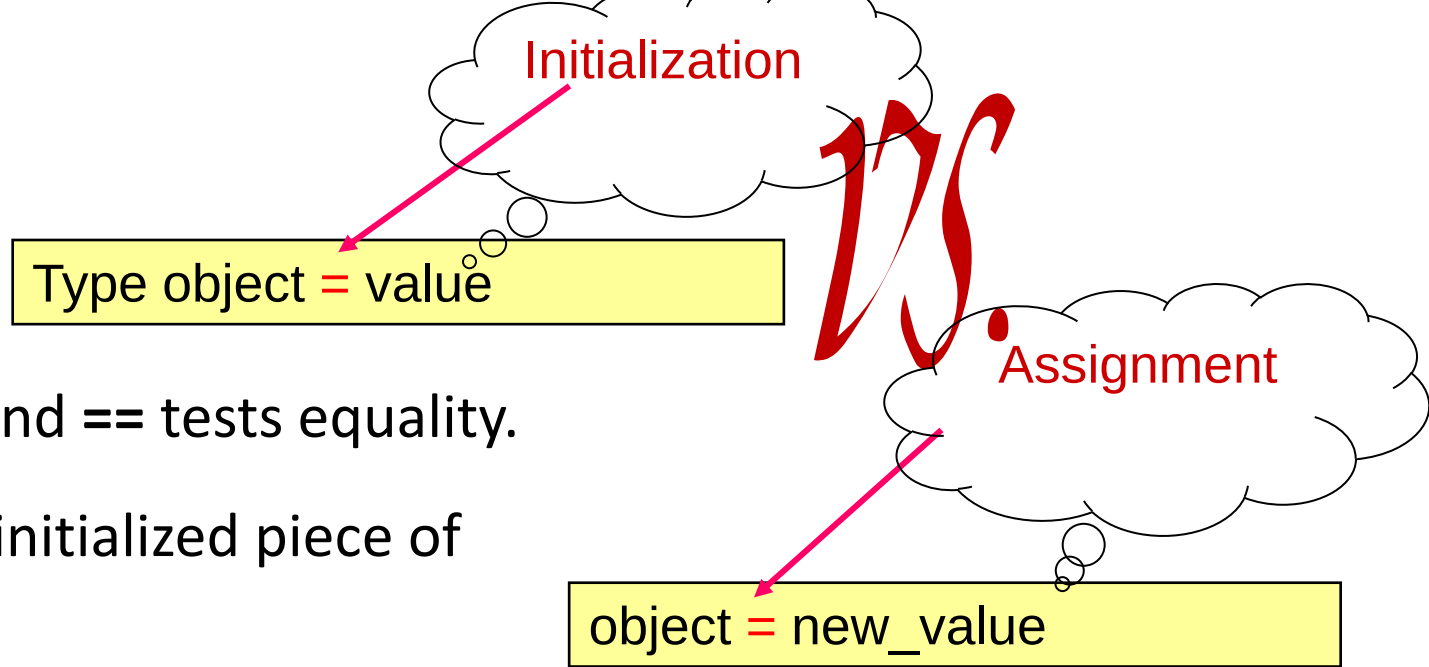
Initialization vs. assignment

- Initialization differs from assignment.
- Note that `=` is the assignment operator and `==` tests equality.
- the task of initialization is to make an uninitialized piece of memory into a valid object.
- Before an object can be used, it must be given a value. C++ offers a variety of notations for expressing initialization, such as the `=` used above, and a universal form based on curly-brace delimited initializer lists.



Initialization vs. assignment

- Initialization differs from assignment.
- Note that = is the assignment operator and == tests equality.
- the task of initialization is to make an uninitialized piece of memory into a valid object.

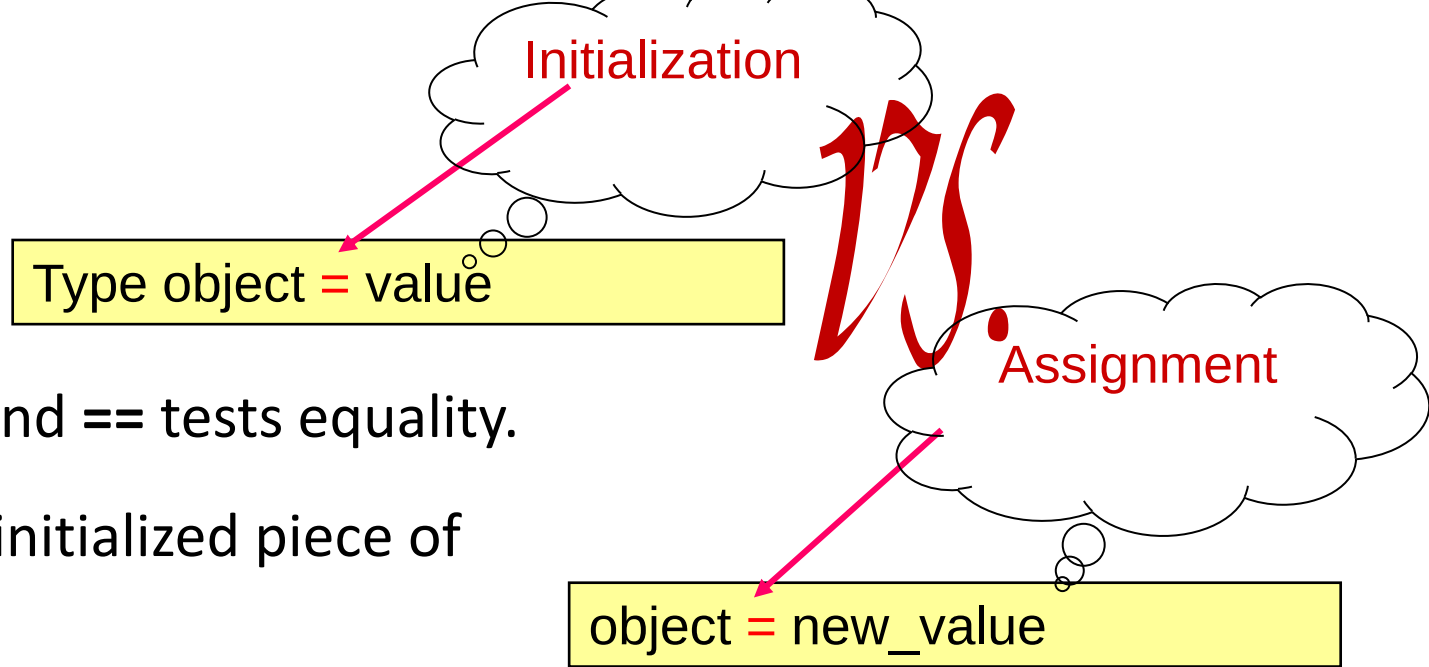


- Before an object can be used, it must be given a value. C++ offers a variety of notations for expressing initialization, such as the = used above, and a universal form based on curly-brace delimited initializer lists.

```
double d1 = 2.3; // initialize d1 to 2.3
double d2 {2.3}; // initialize d2 to 2.3
double d3 = {2.3}; // initialize d3 to 2.3 (the = is optional with { ... })
d3 = d2; // assignment
d3{d1}; // error
```

Initialization vs. assignment

- Initialization differs from assignment.
- Note that = is the assignment operator and == tests equality.
- the task of initialization is to make an uninitialized piece of memory into a valid object.



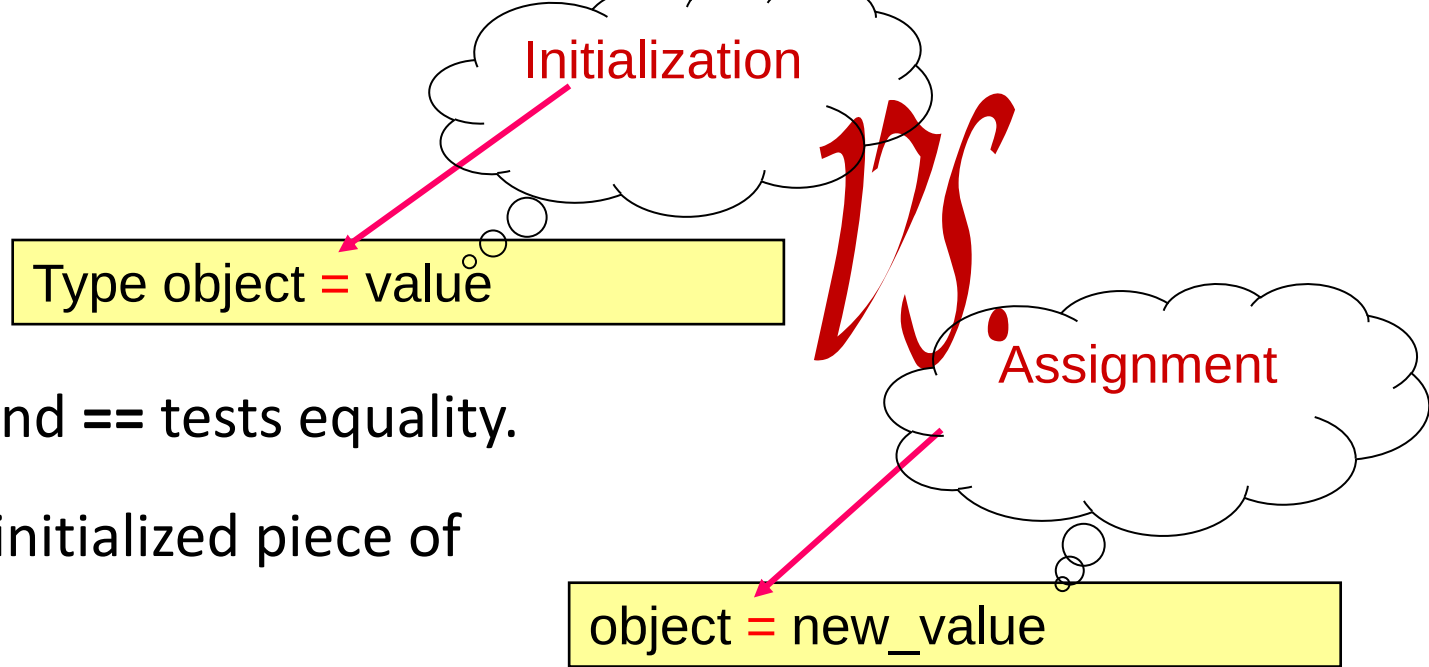
- Before an object can be used, it must be given a value. C++ offers a variety of notations for expressing initialization, such as the = used above, and a universal form based on curly-brace delimited initializer lists.

```
double d1 = 2.3; // initialize d1 to 2.3
double d2 {2.3}; // initialize d2 to 2.3
double d3 = {2.3}; // initialize d3 to 2.3 (the = is optional with { ... })
d3 = d2; // assignment
d3{d1}; // error
```

- User-defined types

Initialization vs. assignment

- Initialization differs from assignment.
- Note that = is the assignment operator and == tests equality.
- the task of initialization is to make an uninitialized piece of memory into a valid object.

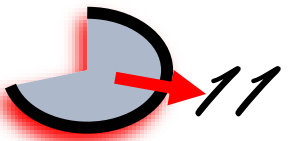


- Before an object can be used, it must be given a value. C++ offers a variety of notations for expressing initialization, such as the = used above, and a universal form based on curly-brace delimited initializer lists.

```
double d1 = 2.3; // initialize d1 to 2.3
double d2 {2.3}; // initialize d2 to 2.3
double d3 = {2.3}; // initialize d3 to 2.3 (the = is optional with { ... })
d3 = d2; // assignment
d3{d1}; // error
```

- User-defined types
 - Initialization: Constructors, → Default, Copy, and Move
 - Assignment: Copy assignment and Move assignment operators

preventing **N**arrowing



- {} initialization doesn't narrow:

```
int i = 7.2; // i becomes 7
int i2{7.2}; // error: double to integer conversion
int i3 = {7.2}; // error: double to integer conversion
double d = 7;
int x2{d};      // error: narrowing (double to int)
char x3{7};     // ok: even though 7 is an int, this is not narrowing
vector<int> vi = { 1, 2.3, 4, 5.6 }; // error: double to int narrowing
```

Auto: Type deduction from initializer

Auto: Type deduction from initializer

1.

Auto: Type deduction from initializer

1.

}

Auto: Type deduction from initializer

1.

Static languages: Fortran, Algol, C, C++, Java, ...

Auto: Type deduction from initializer

1.

Static languages: Fortran, Algol, C, C++, Java, ...

State the type of
variables clearly

Auto: Type deduction from initializer

1.

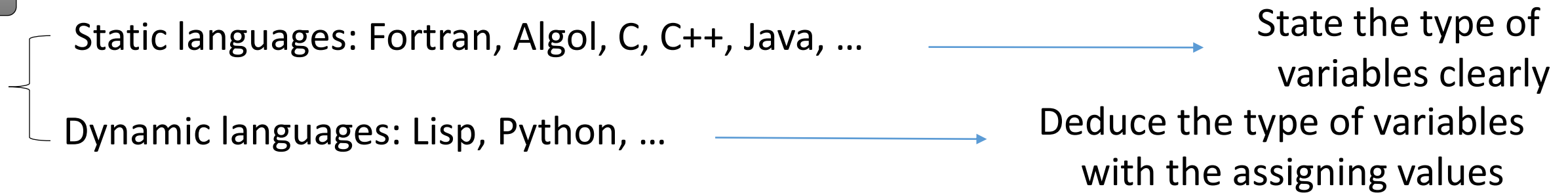
Static languages: Fortran, Algol, C, C++, Java, ...

Dynamic languages: Lisp, Python, ...

State the type of
variables clearly

Auto: Type deduction from initializer

1.



Auto: Type deduction from initializer

1.

Static languages: Fortran, Algol, C, C++, Java, ...

State the type of
variables clearly

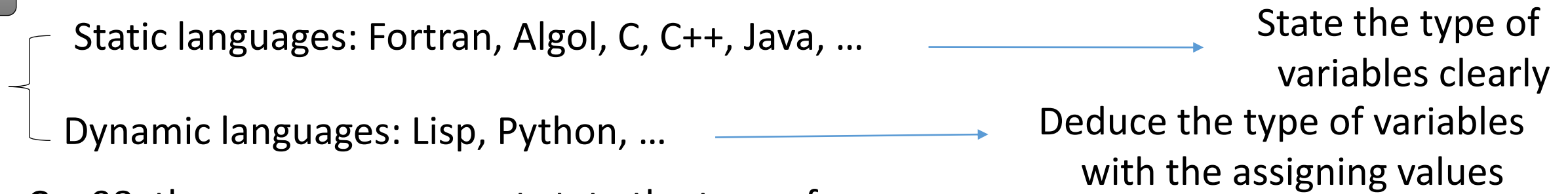
Dynamic languages: Lisp, Python, ...

Deduce the type of variables
with the assigning values

- In C++98, the programmer must state the type of variables.

Auto: Type deduction from initializer

1.



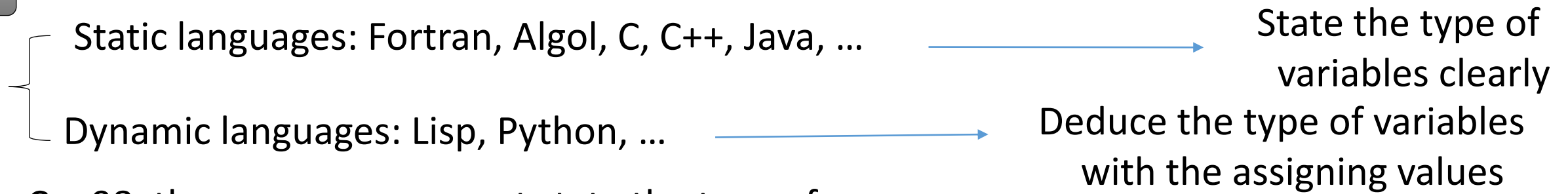
- In C++98, the programmer must state the type of variables.
- Compiler prefers type to initializer.

`<type-specifier> variable-name = initializeropt ;`

```
double g = 9.81; // g is double
int answer = 42.1; // answer = 42
```

Auto: Type deduction from initializer

1.



- In C++98, the programmer must state the type of variables.
- Compiler prefers type to initializer.
- Python

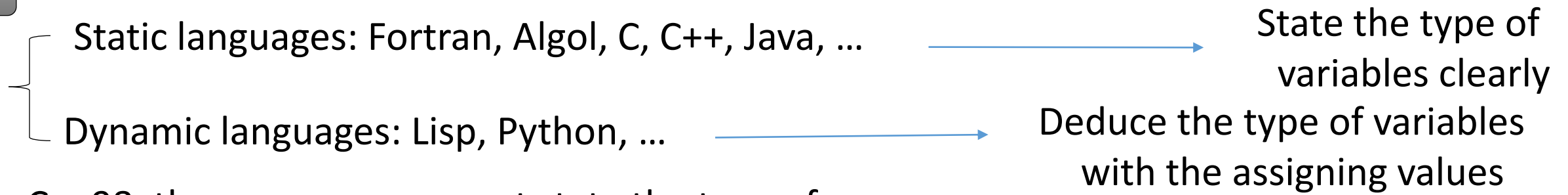
```
answer = 42    # answer is now an int
answer = 42.1  # answer is now a float
```

```
<type-specifier> variable-name = initializeropt ;
```

```
double g = 9.81; // g is double
int answer = 42.1; // answer = 42
```

Auto: Type deduction from initializer

1.



- In C++98, the programmer must state the type of variables.
- Compiler prefers type to initializer.
- Python

```
answer = 42    # answer is now an int
answer = 42.1  # answer is now a float
```

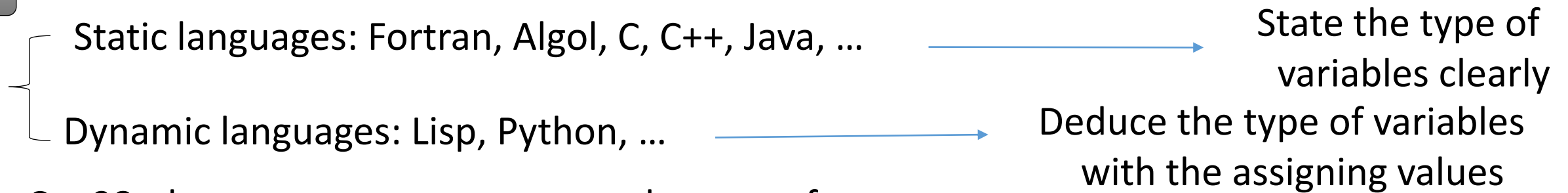
```
<type-specifier> variable-name = initializeropt ;
```

```
double g = 9.81; // g is double
int answer = 42.1; // answer = 42
```

2.

Auto: Type deduction from initializer

1.



- In C++98, the programmer must state the type of variables.
- Compiler prefers type to initializer.
- Python

```
answer = 42    # answer is now an int  
answer = 42.1 # answer is now a float
```

<type-specifier> variable-name = initializer_{opt} ;

```
double g = 9.81; // g is double  
int answer = 42.1; // answer = 42
```

2.

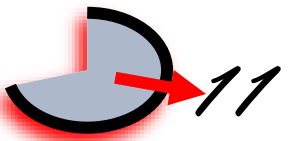
Global variable ←

Local variables ←

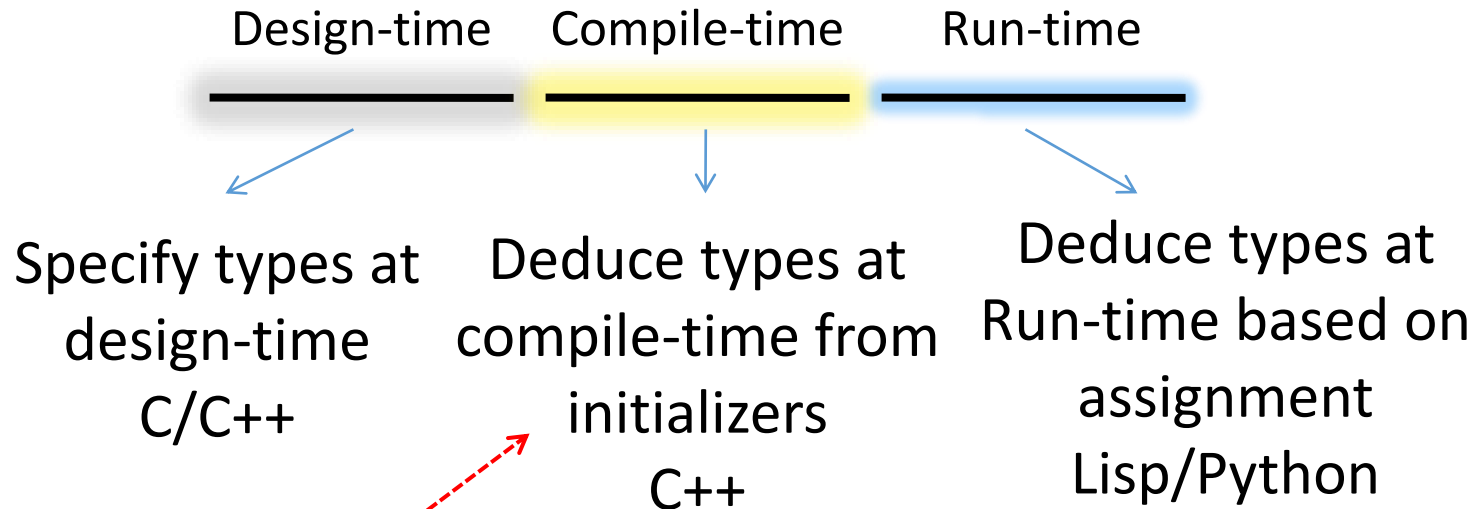
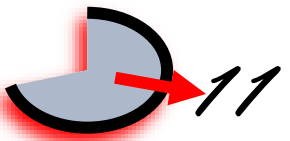
Automatic storage ←

```
bool b; // non-local variable: b = false  
int main()  
{  
    long long LL; // uninitialized  
    auto long long LL; // redundant auto. rather verbose  
    auto std::string s; // calling default ctor  
    return 0;  
} //
```

New mission of auto



New mission of auto



- Third approach: Compiler can figure out the type of expressions at compile-time.
- The original auto is generally useless keyword.
- Recycle the keyword auto:
 - 1) To remove the old redundant usage
 - 2) New responsibility: Determine or deduce the type of expressions from initializer

```
int x = 5; // verbose: the int is redundant  
auto x = 5; // OK: x is int because 5 is int
```

```
auto <variable> = expression
```

Statements

- Statements form the smallest executable unit within a C++ program.
- Statement are terminated with semicolon.
- Null statement: just a single statement

```
; // null statement  
int val;; // additional null statement
```

- Declaration statements
- Expression statements

```
// one of the shortest C++ program  
int main()  
{  
    ;  
}
```

```
float f = 1.0; // declaration statement  
double a = f + 1; // expression statement
```

Selection statements, comparison operators

- Selection statement

- if statement

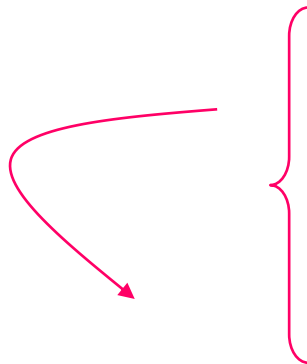
```
if (condition)
    statement
```

```
if (condition)
    statement1
else
    statement2
```

- switch statement

- The condition is an expression that yields a truth value.

Operator	Function	Use
<	less than	expr < expr
<=	less than or equal	expr <= expr
>	greater than	expr > expr
>=	greater than or equal	expr >= expr
==	equality	expr == expr
!=	inequality	expr != expr



Relop

- Relop returns bool (true or false)

```
if (x) // ...
```

```
if (x != 0) // ...
```



Simple computations, simple programs

- Write a program that asks the user to enter two numbers and tells the user which number is larger than the other.
- Declaring multiple names: comma-separated declarators.
- Divide two floating-point numbers. read from input two long double numbers and divide first by second. Note: Division by zero is *mathematically undefined*.



```
// Maximum of two numbers
#include <iostream>
int main()
{
    std::cout << "Enter two numbers: ";
    int a, b; // declaring multiple names
    std::cin >> a >> b;
    int max;
    if (a > b)
        max = a;
    else
        max = b;
    std::cout << "Max = " << max << std::endl;

    return 0;
}
```

```
// floating-point division
#include <iostream>

int main()
{
    std::cout << "Enter two numbers: ";
    long double a, b;
    std::cin >> a >> b;

    // to be continued on next page
}
```

Simple computations, simple programs cont.

```
// continued from last page
if (b == 0.0) { // handle division by zero
    std::cout << "Division by zero!" << '\n';
    return 0;
}
std::cout << a << '/' << b << " = " << a / b << '\n';

return 0;
}
```

- Common mistake:

```
if (a = 7) { // oops!: constant assignment in condition
}
```

```
// most likely it should be
if (a == 7) {
}
```

Compilers usually
issue warning

Rferences and further readings

- [Str00] Bjarne Stroustrup. The C++ Programming Language (Special Edition). Addison Wesley. Reading Mass. USA. February 2000.
- [Str08] Bjarne Stroustrup. Programming: Principles and Practice in C++, 1st edition, Addison Wesley, December 2008.
- [Str13] Bjarne Stroustrup. The C++ Programming Language. 4th edition. Addison Wesley. Reading Mass. USA. May 2013.
- [Str22] Bjarne Stroustrup. A Tour of C++, 3rd edition, Addison Wesley, September 2022.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

